VISVESVARAYA TECHNOLOGICAL UNIVERSITY
"JNANA SANGAMA", BELAGAVI - 590 018

MINI PROJECT REPORT
on

"Student Resource Sharing Platform"

Submitted by

Ganavi G Kotian	**4SF24CS061**
Shukruta Suresh Hanbar	**4SF25CS420**

In partial fulfillment of the requirements for the IV semester
WEB DEVELOPMENT
(Ability Enhancement Course - BCS408A4)

Under the Guidance of

Ms. Alakananda K	**Dr. Poornima B V**
Assistant Professor	**Associate Professor**

BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE & ENGINEERING

SAHYADRI
College of Engineering & Management
An Autonomous Institution
MANGALURU

2025 - 26

SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the mini project work entitled **"Student Resource Sharing Platform"** has been carried out by **Ganavi G Kotian (4SF24CS061) and Shukruta Suresh Hanbar (4SF25CS420)**, the bonafide students of Sahyadri College of Engineering & Management in partial fulfillment of the requirements for the IV semester **Web Development** (AEC course), of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi during the year 2025 - 26. It is certified that all suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Project Guide	**Project Guide**	**HoD**
Ms. Alakananda K	**Dr. Poornima B V**	**Dr. Mustafa Basthikodi**
Assistant Professor	Associate Professor	Professor & Head

External Evaluators **(Name & Signature)**

1.

2.

SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

Department of Computer Science & Engineering

DECLARATION

We hereby declare that the entire work embodied in this Mini Project Report titled **"Student Resource Sharing Platform"** has been carried out by us at Sahyadri College of Engineering & Management, Mangaluru under the supervision of **Ms. Alakananda K and Dr. Poornima B V,** in partial fulfillment of the requirements for the IV semester Web Development (AEC course), of **Bachelor of Engineering** in **Computer Science and Engineering**. This report has not been submitted to this or any other University for the award of any other degree.

Ganavi G Kotian (4SF24CS061)
Shukruta Suresh (4SF24CS420)
Dept. of CSE, SCEM, Mangaluru

Abstract

The **Student Resource Sharing Platform** is a web-based application designed to facilitate the efficient sharing and management of academic resources among students. The primary objective of this system is to create a centralized platform where students can upload, access, and download study materials such as notes, assignments, question papers, and reference documents.

The platform is developed using the MERN stack, which includes MongoDB for database management, Express.js and Node.js for backend development, and React for building an interactive user interface. The system provides user authentication to ensure secure access, allowing registered users to upload and manage their resources efficiently.

Key features of the platform include categorized resource storage based on subjects or departments, a powerful search functionality to quickly locate required materials, and options for downloading and sharing files. Additionally, the system may include features such as user ratings, comments, and file validation to maintain the quality and relevance of shared content.

This project aims to enhance collaborative learning, reduce duplication of effort, and provide easy access to academic materials anytime and anywhere. It serves as a practical solution to improve knowledge sharing among students while demonstrating the implementation of modern web technologies in real-world applications.

Table of Contents

Chapter 1

Introduction

1.1 Overview

Every student has been there — it's the night before an exam, you need notes for a topic you missed, and you spend more time searching for them than actually studying. You check WhatsApp groups, ask friends, dig through old emails, and still end up with incomplete material. This is a problem that almost every college student faces, and surprisingly, there's no simple solution built specifically for students.

SRSP was created to be that solution. It's a platform where students can upload their notes, assignments, and question papers, and where other students can find, preview, and download them — organized by department, subject, and category. No more hunting through chat groups. No more "can you send me the notes?" messages. Everything is in one place, searchable, and accessible anytime.

The application is built on the MERN stack — MongoDB handles the database, Express.js and Node.js power the backend API, and React builds the frontend interface. MongoDB stores all user accounts and resource data in a flexible document format. Express and Node handle everything on the server side — authentication, file uploads, API requests, and data retrieval. React delivers a fast, responsive interface that works on any modern browser without page reloads.

Security is handled through JSON Web Tokens. Only registered students can upload content, which ensures accountability and helps maintain the quality of resources on the platform. The interface is designed to be simple enough that any student can register, upload a file, and have it available to their peers within minutes — no technical knowledge required.

In short, SRSP brings together document sharing, community feedback, and organized storage into a single, easy-to-use platform built around how students actually work.

1.2 Scope & Motivation

Scope:

- SRSP is designed specifically for use within a college or university. It covers:
- Student registration and login with JWT-based authentication
- File uploads in multiple formats — PDF, DOCX, PPTX, images, ZIP
- Resource browsing with search, department filter, and category filter
- In-browser file preview — no download needed just to check if something is useful
- Star ratings and comments for community-driven quality control
- A personal dashboard where students can manage and delete their own uploads
- Download tracking so uploaders can see how much their contributions are being used

Motivation:

The motivation behind SRSP comes from a very real frustration. In most colleges, there's no official channel for students to share study materials with each other. Notes get shared in WhatsApp groups — but only with people in that group. Question papers from previous years circulate informally among students who know the right seniors. Good reference material stays within small circles and never reaches the students who need it most.

There's also the issue of equity. Not every student has access to the same resources — some can afford coaching and reference books, some can't. When one student uploads good notes, every student on the platform benefits regardless of their background or social circle. That's the core motivation behind SRSP — making sure that access to quality academic material isn't determined by who you know or how much you can spend.

Beyond that, there's the problem of knowledge preservation. Every year, graduating students leave and take their notes with them. Years of carefully prepared study material simply disappears. SRSP creates a permanent, searchable repository where that knowledge stays accessible to future batches long after the original student has graduated.

Chapter 2

Literature Survey

A literature survey was conducted to understand existing platforms, research, and technologies related to academic resource sharing, collaborative learning systems, and MERN stack web development. The following table summarizes the most relevant and recent references reviewed during the development of SRSP.

Table 2.1: Literature Review for Student Resource Sharing Platform

Sl. No.	Name	Authors	Key Findings	Gaps
1.	"Collaborative Learning Through Digital Platforms in Higher Education"	Sharma, R. & Patel, S. (2019) — IEEE Transactions on Education	Peer-to-peer knowledge sharing improves academic outcomes by over 20%. Students who actively share resources perform better in assessments. Digital platforms significantly increase engagement in collaborative learning.	The study focuses on outcomes but does not propose or evaluate a specific platform for resource sharing. No implementation details are provided.
2.	"Google Classroom as a Learning Management Tool"	Iftakhar, S. (2016) — Journal of Education and Practice	Google Classroom simplifies assignment distribution and communication between teachers and students. It is widely adopted and easy to use.	Entirely teacher-centric. Students cannot upload or share resources with peers. Not suitable for informal, student-driven knowledge sharing.
3.	"Moodle-Based E-Learning: A Study on Student Engagement"	Al-Ajlan, A. & Zedan, H. (2018) — IEEE DEST Conference	Moodle provides a comprehensive LMS with forums, quizzes, and file sharing.	Requires institutional setup and administrator management. Students cannot

			Institutions using Moodle report higher student engagement compared to traditional methods.	independently register and share content. Too complex for lightweight peer sharing.
4.	"Document Sharing Platforms in Academic Environments: A Comparative Study"	Kumar, A. & Verma, N. (2020) — International Journal of Computer Applications	Platforms like Scribd and SlideShare allow broad document sharing but lack academic categorization. Users struggle to find subject-specific or institution-specific content.	No authentication or accountability system. No department or subject-based filtering. Not designed for college-specific use cases.
5,	"MERN Stack for Developing Scalable Web Applications"	Patel, D. & Shah, M. (2021) — International Journal of Engineering Research and Technology	MERN stack offers faster development cycles, better performance for data-heavy applications, and easier scalability compared to traditional PHP/MySQL stacks. React's component model improves UI maintainability.	The paper focuses on general MERN development. No application to academic resource sharing or student-specific use cases is explored.
6.	"JWT-Based Authentication for Secure Web Applications"	Rathore, S. & Gupta, P. (2020) — Journal of Network and Computer Applications	JWT provides a stateless, scalable authentication mechanism suitable for REST APIs. Tokens reduce server-side session management overhead and improve security in distributed	The paper covers authentication in general web contexts. Integration with student-specific role-based access and college email validation is not addressed.

			systems.	
7.	"File Upload and Management in Node.js Applications Using Multer"	Singh, A. (2022) — International Journal of Advanced Computer Science	Multer efficiently handles multipart form data in Node.js. It supports file type validation, size limits, and custom storage configurations. Performance remains stable for files up to 50MB.	The study focuses on technical implementation only. No discussion of how file management integrates with academic platforms or user experience considerations
8.	"MongoDB for Modern Web Applications: Performance and Flexibility"	Chen, L. & Wang, Y. (2021) — IEEE Access	MongoDB's document model is well-suited for applications with nested or variable data structures. It outperforms relational databases in read-heavy workloads and scales horizontally with ease.	No specific evaluation of MongoDB for academic platforms with embedded arrays like ratings and comments. Schema design for such use cases is not discussed.
9.	"The Role of Student-Generated Content in Higher Education"	Bruns, A. & Humphreys, S. (2019) — Computers & Education	When students create and share content, they develop deeper understanding of the subject matter. Platforms that encourage student-generated content improve both the contributor's and the reader's learning outcomes.	The research is theoretical. No practical platform is built or evaluated. The findings are not connected to any specific technology implementation.
10.	"React.js for Building Modern Single Page Applications"	Fedosejev, A. (2020) — Packt Publishing	React's virtual DOM and component-based architecture significantly	Focuses on React in isolation. No discussion of integrating React with a full

			improve UI performance and code reusability. State management through hooks simplifies complex UI interactions.	backend stack for academic or resource-sharing applications.
11	"Cloud Databases in Education: MongoDB Atlas as a Case Study"	Gupta, R. & Mehta, K. (2022) — Journal of Cloud Computing	MongoDB Atlas provides reliable, always-on cloud database hosting with built-in security, automatic backups, and global availability. Free tier is sufficient for small to medium academic applications.	The paper evaluates Atlas from an infrastructure perspective only. No discussion of how cloud databases support real-time academic collaboration features.
12.	"Digital Equity in Education: Bridging the Resource Gap Among Students"	UNESCO Report (2021)	Students from lower-income backgrounds consistently have less access to quality study materials. Digital platforms that provide free, open access to academic resources can significantly reduce this gap.	The report identifies the problem and recommends digital solutions but does not propose or evaluate any specific platform or technical implementatio

Chapter 3

Problem Formulation

3.1 Problem Description

Walk into any college and ask students how they share study materials. The answer is almost always some combination of WhatsApp groups, Google Drive links, and personal favors. It works, barely, but it creates real problems. Materials are hard to find, quality is inconsistent, and there's no way to know if what you're downloading is actually useful until you've already downloaded it.

Senior students graduate and take their notes with them. Good resources get shared in small circles and never reach the students who need them most. There's no system, no organization, and no accountability.

3.2 Problem Statement

To design and develop a web-based Student Resource Sharing Platform that allows authenticated college students to upload, browse, preview, download, rate, and comment on academic resources — organized by department and category — using the MERN stack.

3.3 Objectives

- Build a secure registration and login system so only college students can access the platform
- Allow students to upload files in common academic formats with proper metadata
- Organize resources by department, subject, and category for easy discovery
- Enable in-browser file preview so students can check content before downloading
- Implement search and filter so finding the right resource takes seconds, not minutes
- Add ratings and comments so the community can identify the best resources
- Give students control over their own uploads — view, manage, and delete
- Make the interface clean, fast, and easy to use without any training

3.4 Functional Requirements

ID	Requirement
FR1	Students can register with name, student ID, email,

	department, year, and password
FR2	Registered students can login using email and password
FR3	Logged-in students can upload files with title, subject, department, and category
FR4	All visitors can browse and search resources
FR5	Users can filter resources by department and category
FR6	Users can preview PDF and document files directly in the browser
FR7	Users can download any resource
FR8	Logged-in users can rate resources from 1 to 5 stars
FR9	Logged-in users can post comments on resources
FR10	Students can delete their own uploaded resources

3.5 Non-Functional Requirements

ID	Requirement
NFR1	API responses should be delivered within 2 seconds under normal conditions
NFR2	Passwords must be hashed using bcrypt — never stored as plain text
NFR3	All upload, comment, rate, and delete actions require a valid JWT token
NFR4	The platform should support file sizes up to 20MB
NFR5	The interface should work well on both desktop and mobile browsers
NFR6	MongoDB schema validation should prevent malformed data from being saved
NFR7	Using MongoDB Atlas ensures the database is available 24/7 from the cloud

Chapter 4

Project Design and Implementation

4.1 High Level Design

SRSP follows a standard three-tier web architecture — the browser talks to the React frontend, which communicates with the Express backend via REST APIs, which in turn reads and writes to MongoDB Atlas.

The frontend is a React single-page application. When a student opens the app, React handles all the page navigation without full page reloads. Every action that needs data — loading resources, logging in, uploading a file — goes through an API call to the Express backend. The backend processes the request, talks to MongoDB, and sends back a response. The frontend then updates what the student sees.

Authentication works through JWT tokens. When a student logs in, the server issues a token that gets stored in the browser. Every subsequent request that needs authentication includes this token in the header. The server checks it before allowing the action.

File uploads go through Multer, a Node.js middleware that intercepts the file from the request and saves it to the server's uploads folder. The file's metadata — title, subject, department, category — gets saved to MongoDB along with the filename so it can be retrieved later.

4.2 Detailed Design

Database Collections:

Users — stores all registered student accounts:

Field	Type	Description
name	String	Student's full name
studentId	String	Unique college ID
email	String	College email, used for login
department	String	e.g. Computer Science
year	String	e.g. 3rd Year
password	String	bcrypt hashed password

Resources — stores all uploaded materials:

Field	Type	Description
title	String	Resource title
description	String	Optional description
subject	String	e.g. Data Structures
department	String	e.g. Computer Science
category	Enum	Notes / Assignment / Question Paper / Reference
filename	String	Server-side stored filename
originalName	String	Original uploaded filename
uploadedBy	ObjectId	Reference to User
downloads	Number	Increments on each download
ratings	Array	Each entry: user ID + star value
comments	Array	Each entry: user ID, name, text, date

API Endpoints:

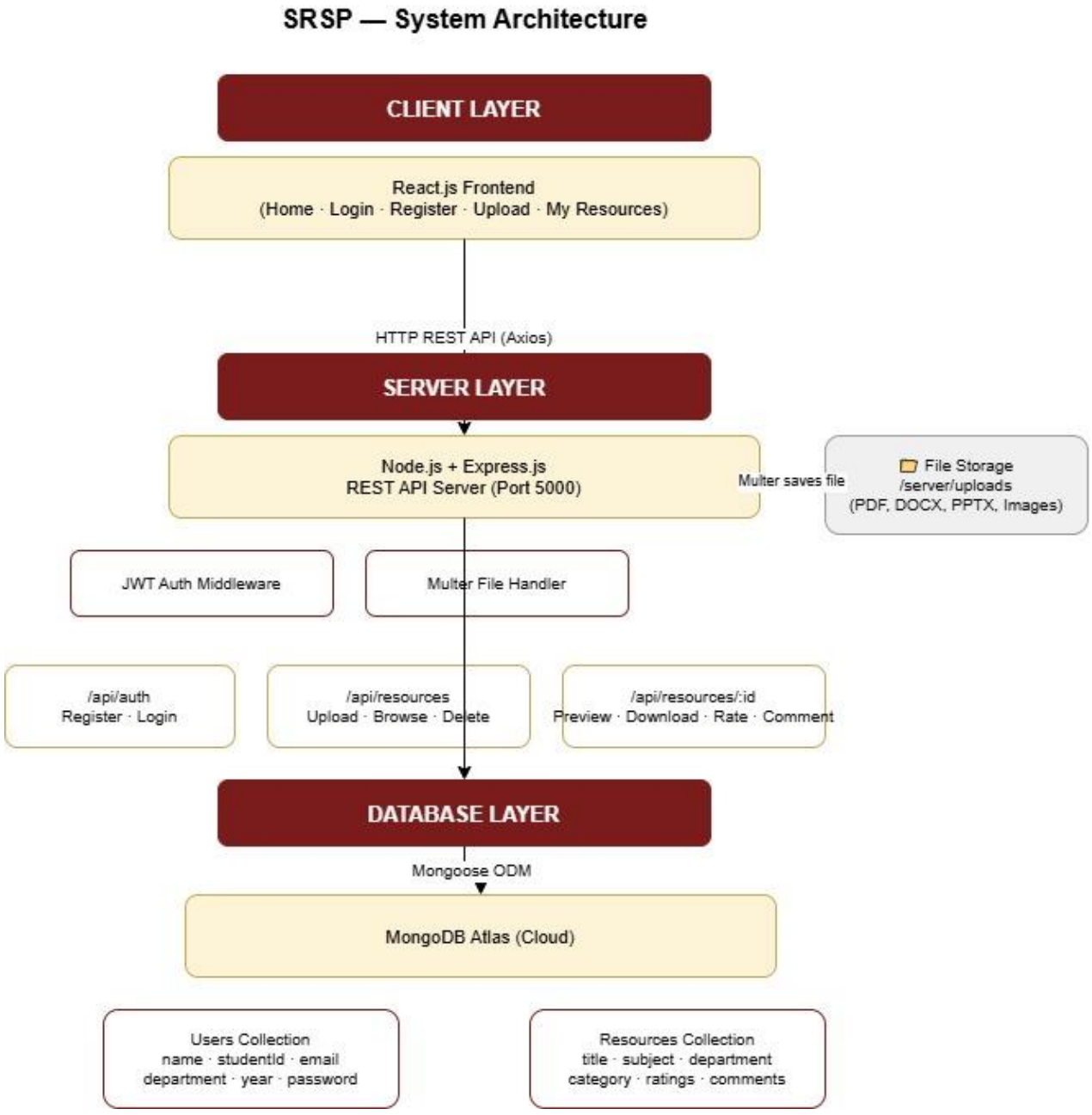
Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Register a new student
POST	/api/auth/login	No	Login and receive JWT
GET	/api/resources	No	Fetch all resources with optional filters
POST	/api/resources	Yes	Upload a new resource
GET	/api/resources/:id/preview	No	View file inline in browser
GET	/api/resources/:id/download	No	Download the file
GET	/api/resources/:id/comments	No	Fetch all comments

POST	/api/resources/:id/comment	Yes	Post a comment
POST	/api/resources/:id/rate	Yes	Rate a resource
DELETE	/api/resources/:id	Yes	Delete own resource

4.3 Tools and Technologies Used

Category	**Technology**	**Purpose**
Frontend Framework	React.js 18	Building the user interface
Client Routing	React Router DOM 6	Navigation between pages
HTTP Requests	Axios	Calling backend APIs from React
Backend Runtime	Node.js 16+	Running JavaScript on the server
Backend Framework	Express.js 4	Building the REST API
Database	MongoDB Atlas	Cloud-hosted NoSQL database
ODM	Mongoose 7	Defining schemas and querying MongoDB
Authentication	JSON Web Token	Stateless user sessions
Password Security	bcryptjs	Hashing passwords before storage
File Uploads	Multer	Handling multipart form data
Dev Utility	Nodemon	Auto-restarting server on file changes
Document Preview	Google Docs Viewer	Previewing DOCX and PPTX in browser

4.4 Architecture Diagram



Chapter 5

Results and Discussion

5.1 Experimentation Environment Setup

Hardware used:

- Laptop with Intel Core i5 processor
- 8GB RAM
- Standard broadband internet connection (required for MongoDB Atlas)

Software environment:

- Windows 10/11
- Node.js v16+
- Google Chrome browser
- Visual Studio Code
- MongoDB Atlas free tier (cloud)

To run the project:

Step 1 — Configure the database by editing

server/.env and setting the MongoDB Atlas connection string and JWT secret key.

Step 2 — Install dependencies by running

npm install inside both the /server and /client folders.

Step 3 — Start the backend in one terminal with

cd server && npm run dev. The server starts on port 5000.

Step 4 — Start the frontend in a second terminal with

cd client && npm start. The app opens at http://localhost:3000.

5.2 Front End GUI Snapshots

The home page is the first thing students see. The top section has the platform name, a short description, and four stat boxes showing total resources, downloads, departments, and students. Below that is a search bar where students can type any keyword, and two dropdowns to filter by department and category. The resources appear as cards in a grid, each showing the title, subject, department, category badge, uploader name, download count, and star rating.

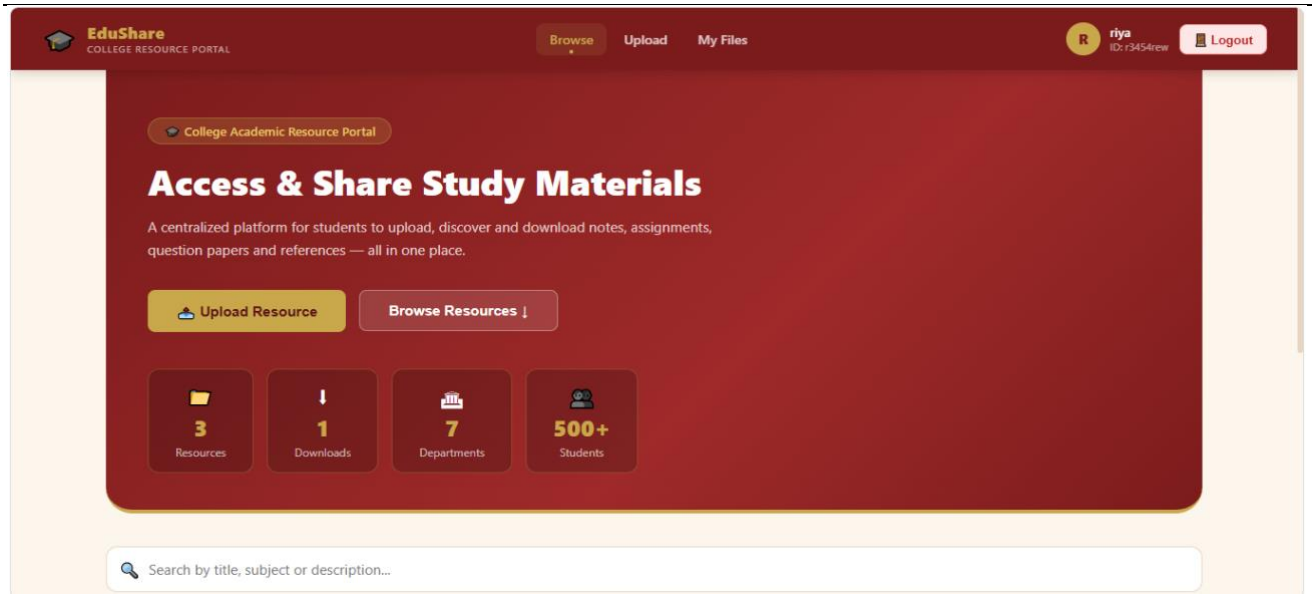


Fig5.1:Home Page / Browse Resources

The registration page has a split layout. The left side explains the platform in brief. The right side has the form — students fill in their full name, student ID, college email, department, year, and password. All fields are validated before submission. Once registered, the student is automatically logged in and redirected to the home page.

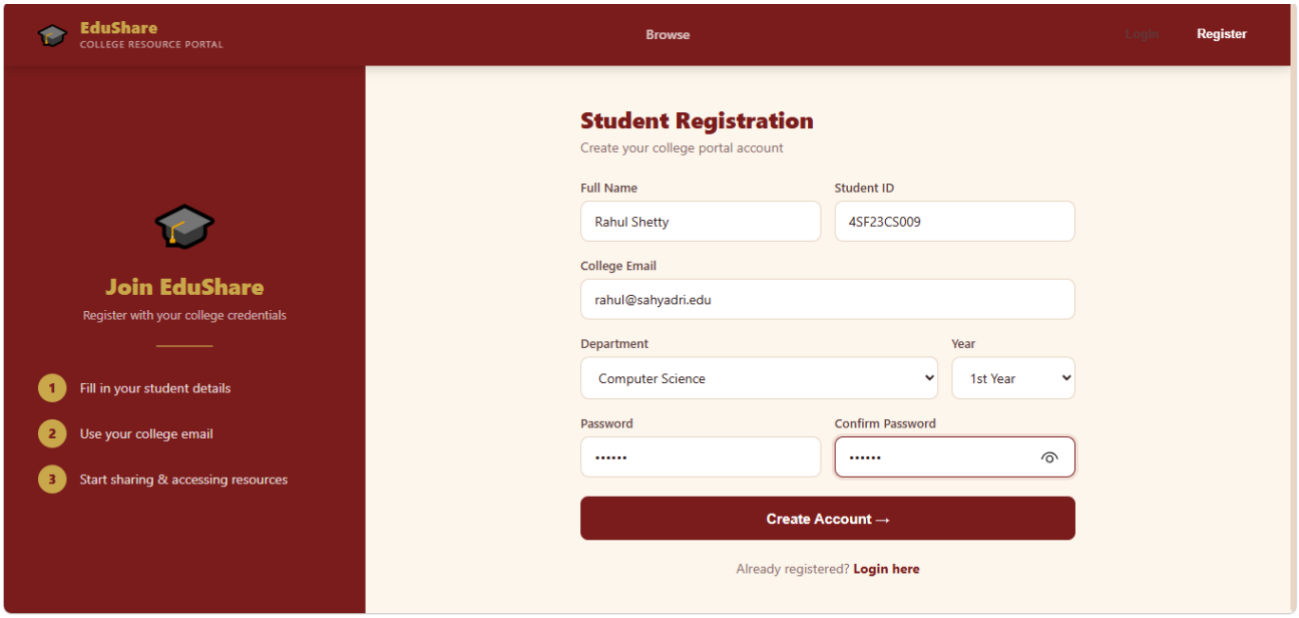


Fig 5.2:Register Page

The login page follows the same split layout. Students enter their registered email and password. If the credentials are wrong, a clear error message appears. On successful login, the navbar updates to

show the student's name and student ID, and the Upload and My Files options become visible.

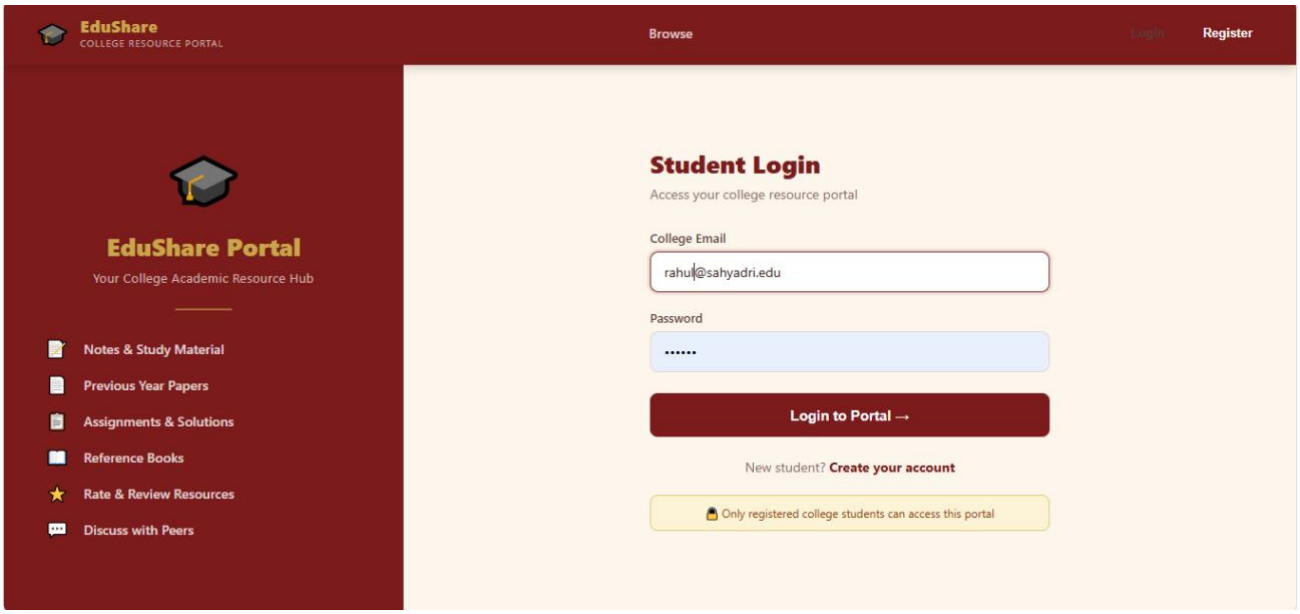


Fig 5.3: Login Page

The upload page has a clean two-column layout. The main form on the left lets students fill in the resource title, description, subject, select a department, pick a category by clicking on visual cards (Notes, Assignment, Question Paper, Reference), and upload a file by dragging and dropping or clicking to browse. The sidebar on the right shows who is uploading and a few helpful tips.

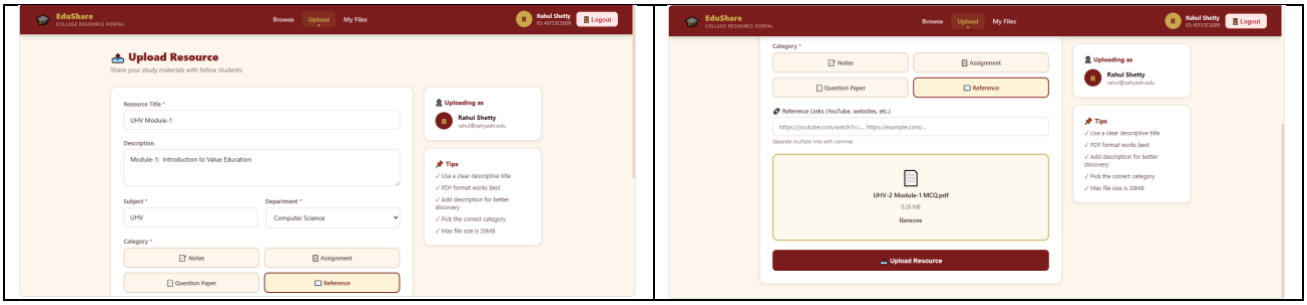


Fig:5.4 : Upload Page

Each resource card has a Comments button that expands a section below showing all comments posted by students. Logged-in users see a text input to add their own comment. Comments are fetched fresh from the database every time the section is opened, so comments from other accounts appear immediately.

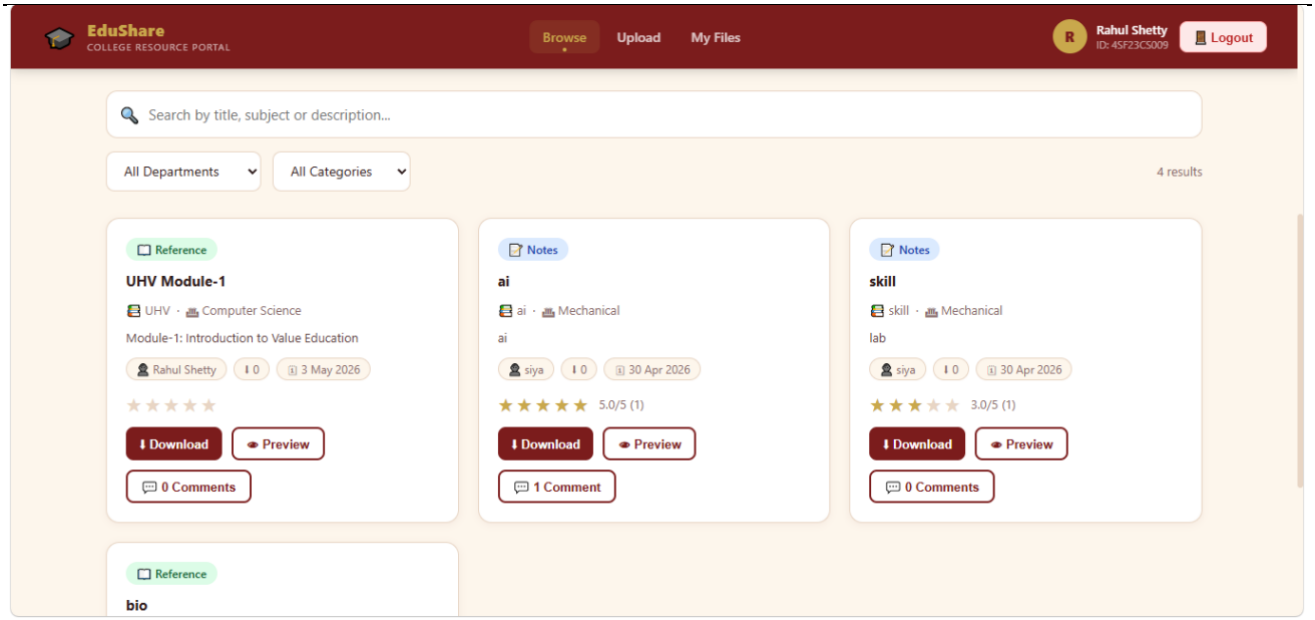


Fig 5.5 : Resource Card with Comments

Clicking the Preview button on any resource opens a full-screen modal. For PDF files, the document renders directly inside the modal using an embedded viewer — no download required. For DOCX and PPTX files, Google Docs Viewer handles the rendering. The modal has a close button at the top right.

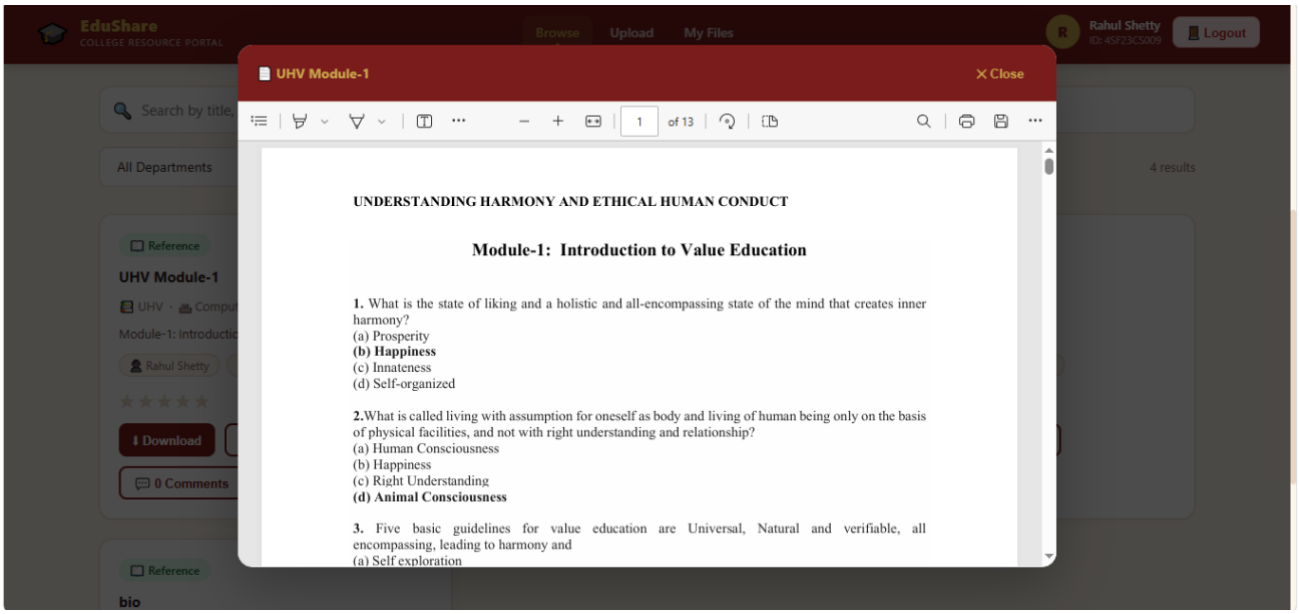


Fig 5.6 : File Preview Model

The My Resources page shows everything the logged-in student has uploaded. At the top are four stat cards — total uploads, total downloads, average rating, and total comments received. Below that, each

resource card has a red delete button. Clicking it asks for confirmation before permanently removing the resource.

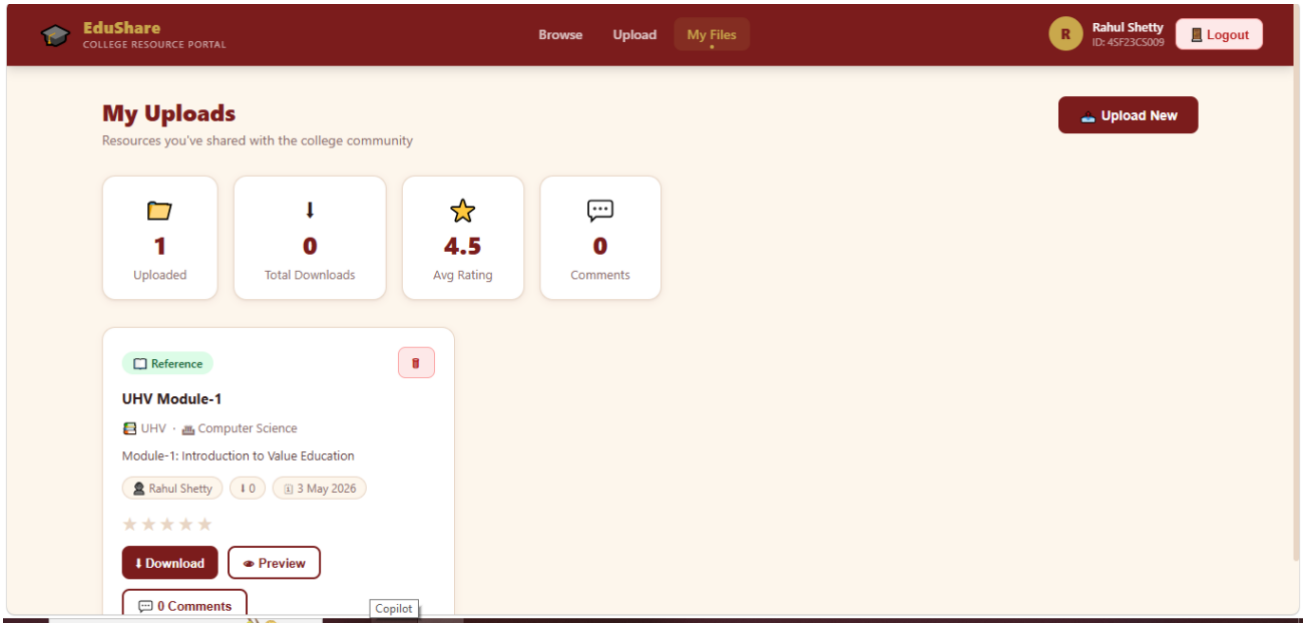


Fig 5.7 : My Resources Page

The search and filter system works in real time. Typing a keyword filters resources by title, subject, or description. Selecting a department or category from the dropdowns narrows the results further. A result count below the filters shows how many resources match the current search.

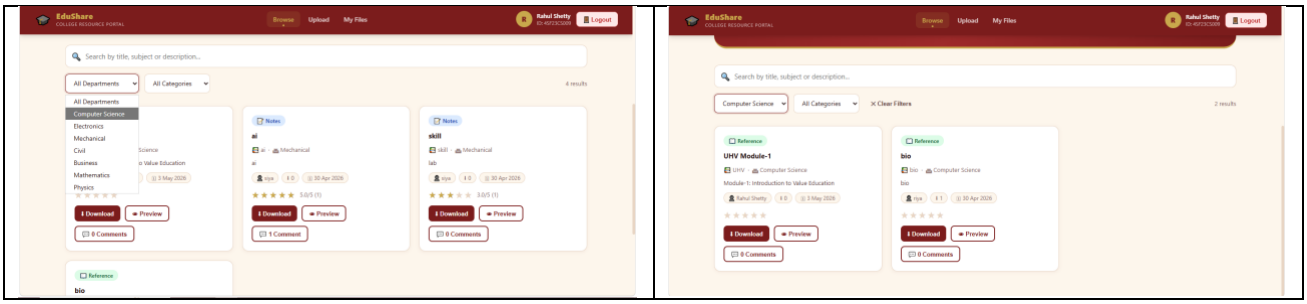


Fig 5.8 : Search and Filter in Action

5.3 Back End Description

The backend is a Node.js and Express.js REST API with the following key components:

Authentication: When a student registers, their password is hashed with bcrypt (10 salt rounds) and stored in MongoDB. Plain text passwords are never saved. On login, the submitted password is compared against the stored hash using bcrypt's compare function. If it matches, a JWT token is signed

with the student's ID and name, valid for 7 days. This token is returned to the frontend and stored in the browser's localStorage. Every protected request includes this token in the Authorization header as a Bearer token. The auth middleware on the server verifies the token's signature and expiry before allowing the request to proceed.

File handling: Multer intercepts file upload requests before they reach the route handler. It saves the uploaded file to the

/server/uploads directory with a timestamp prefix added to the filename to prevent naming conflicts. Both the original filename and the server filename are stored in MongoDB — the original name is used when serving the file for download so the student gets a properly named file.

File preview: For preview requests, the server reads the file extension, sets the appropriate Content-Type header (application/pdf for PDFs, image/jpeg for JPEGs, etc.), and sends the file with

Content-Disposition: inline. This tells the browser to display the file rather than download it. For DOCX and PPTX files, the frontend passes the file URL to Google Docs Viewer which handles rendering in an iframe.

Comments and ratings: Both are stored as arrays embedded within the resource document in MongoDB. When a comment is posted, it is pushed to the array and the full updated array is returned to the frontend. When a student opens the comments section, a fresh GET request is made to the database — this ensures comments posted by other accounts are always visible, not just the ones loaded when the page first opened.

5.4 Societal Impact of the Project

SRSP has real potential to make a difference in how students learn and collaborate.

The most direct impact is on access to education. Not every student can afford coaching classes or expensive reference books. When one student uploads good notes, every student on the platform benefits regardless of their financial background. That's a meaningful step toward educational equity.

There's also an impact on collaborative culture. Most academic environments are competitive — students guard their notes and don't share. SRSP flips that dynamic by making sharing the default. When students see that their uploads help others and get rated and appreciated, it builds a culture of generosity and mutual support.

From an environmental perspective, digital sharing reduces the demand for printed notes and photocopied materials — a small but real contribution to reducing paper waste on campus.

For senior students, the platform creates a legacy. Notes and resources they upload remain accessible

to future batches, preserving institutional knowledge that would otherwise be lost when they graduate.

5.5 SDG Mapped

SDG	Goal Title	How SRSP Contributes
SDG 4	Quality Education	Provides free, organized access to academic resources for all students, directly improving learning quality and equity
SDG 9	Industry, Innovation and Infrastructure	Demonstrates practical use of modern web technologies to build digital infrastructure for education
SDG 10	Reduced Inequalities	Ensures students from all backgrounds have equal access to study materials, reducing the advantage gap
SDG 17	Partnerships for the Goals	Encourages a culture of collaboration and knowledge sharing among students

Conclusion and Future Scope

Conclusion:

SRSP started as a solution to a simple, everyday problem — students struggling to find and share study materials. What came out of it is a fully functional web application that handles authentication, file management, search, preview, ratings, and comments, all tied together with a clean and intuitive interface.

The MERN stack proved to be the right choice. MongoDB's flexible document model made it easy to store complex resource data with nested ratings and comments. Express and Node handled the API cleanly. React made the frontend fast and responsive. The combination worked well and the

development process validated why this stack is so popular for modern web applications.

More importantly, the platform works. Students can register, upload their materials, and have them instantly available to their peers. Others can find what they need in seconds using search and filters, preview it before downloading, and leave feedback for the community. That's the core loop, and it works end to end.

Future Scope:

- Mobile app — A React Native version for Android and iOS so students can access resources on their phones
- Smart recommendations — Suggest resources based on a student's department, year, and browsing history
- Video support — Allow YouTube links and video uploads for lecture recordings and tutorials
- Notifications — Alert students when new resources are uploaded in their department or subject
- Admin panel — A dashboard for college administrators to moderate content and view usage analytics
- Plagiarism detection — Check uploaded documents for copied content to maintain quality
- Discussion forums — Subject-wise Q&A boards where students can ask and answer questions
- Offline access — Progressive Web App support so downloaded resources can be accessed without internet
- Resource versioning — Let uploaders update their resources while keeping the version history
- Analytics for uploaders — Show how many people viewed, downloaded, and rated each resource

References

[1] MongoDB. (2024). MongoDB Atlas Documentation. https://www.mongodb.com/docs/atlas/

[2] Meta Open Source. (2024). React Documentation. https://react.dev/

[3] OpenJS Foundation. (2024). Express.js Documentation. https://expressjs.com/

[4] OpenJS Foundation. (2024). Node.js Documentation. https://nodejs.org/docs/

[5] Auth0. (2024). Introduction to JSON Web Tokens. https://jwt.io/introduction/

[6]expressjs/multer. (2024). Multer — Node.js middleware for multipart/form-data. https://github.com/expressjs/multer

[7] Mongoose. (2024). Mongoose ODM Documentation. https://mongoosejs.com/docs/

[8] Remix Software. (2024). React Router Documentation. https://reactrouter.com/

[9] Axios. (2024). Axios HTTP Client Documentation. https://axios-http.com/docs/intro

[10]United Nations. (2015). The 2030 Agenda for Sustainable Development. https://sdgs.un.org/goals

[11] Google. (2024). Google Docs Viewer. https://docs.google.com/viewer

[12] dcodeIO. (2024). bcryptjs. https://github.com/dcodeIO/bcrypt.js

[13] Sharma, R., & Patel, S. (2019). Peer Knowledge Sharing in Higher Education: Impact on Academic Outcomes. IEEE Transactions on Education, 62(3), 145–152.

[14] Kumar, A. (2020). Full Stack MERN Development. Packt Publishing.